LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

OLEH:

Nama	:Zafir Amal Maulana
Nim	:2025573010141
Kelas	: TI 1\C
Jurusan	: Teknologi Informasi dan Komputer
Dosen	: M. Reza Zulman S,ST.M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE

2026

LEMBAR PENGESAHAN

Nama Mata Kuliah	: Algoritma Dan Sruktur Data
Judul Pratikum	: Praktikum Analisis Kompleksitas Algoritma
Hari/Tanggal Pratikum	: 22 April 2026
Tanggal Penyerahan	: 28 April 2026
Tempat	: Laboratorium (TIK.106)
Nama Mahasiswa	: Zafir Amal Maulana
NIM	: 2025573010141
Program Studi/Kelas	: Teknik Informatika / TI.1C

Mengetahui
Dosen Pengajar,

M. Reza Zulman S,ST.M.Sc

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
KATA PENGANTAR iii
BAB I PENDAHULUAN 1
1.1 Latar belakang 1
1.2 TujuanPraktikum 2
BAB II LANDASAN TEORI 3
2.1 Analisis Kompleksitas Algoritma 3
2.2 Big O Notation 4
2.3 Time Complexity & Space Complexity 4
2.4 Fungsi & Kegunaan 5
BAB III METODOLOGI PRAKTIKUM 6
3.1 Alat Dan Bahan 6
3.2 Langkah – Langkah 6
BAB IV PENUTUP 20
4.1 KESIMPULAN 20
4.2 SARAN 21

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum yang berjudul “Praktikum Analisis Kompleksitas Algoritma” dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban penulis dalam melaksanakan kegiatan praktikum serta untuk memenuhi tugas yang telah diberikan.

Laporan ini bertujuan untuk mempelajari dan memahami konsep analisis kompleksitas algoritma dalam pengolahan data dan pemecahan masalah. Melalui praktikum ini, penulis dapat mengetahui teknik dalam mengukur efisiensi suatu algoritma, baik dari segi waktu eksekusi maupun penggunaan memori, sehingga dapat memilih solusi yang paling optimal.

Selain itu, praktikum ini juga memberikan pemahaman mengenai konsep-konsep dasar analisis kompleksitas seperti kompleksitas waktu (time complexity), kompleksitas ruang (space complexity), serta penggunaan notasi seperti Big O, Big Theta, dan Big Omega. Penulis juga mempelajari bagaimana menganalisis struktur algoritma seperti perulangan (loop), percabangan, dan rekursi untuk menentukan tingkat efisiensinya.

Praktikum ini juga membantu penulis memahami pentingnya penggunaan algoritma yang efisien, terutama dalam menangani data dalam jumlah besar. Dengan melakukan analisis kompleksitas, proses pengembangan program menjadi lebih terstruktur, optimal, dan mampu menghasilkan performa yang lebih baik.

Melalui praktikum ini, penulis mendapatkan tambahan wawasan dan keterampilan dalam bidang analisis algoritma, khususnya dalam menentukan dan membandingkan kompleksitas suatu algoritma. Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan baik dari segi penulisan maupun isi materi yang disampaikan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari berbagai pihak demi perbaikan dan penyempurnaan laporan ini di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat, menambah wawasan, serta menjadi referensi bagi pembaca yang ingin mempelajari lebih lanjut mengenai analisis kompleksitas algoritma.

BAB I
PENDAHULUAN

1.1 Latar belakang

Perkembangan teknologi informasi yang semakin pesat telah membawa perubahan besar dalam berbagai bidang, khususnya dalam pengolahan data dan penyelesaian masalah berbasis komputasi. Dalam dunia ilmu komputer, algoritma memegang peranan penting sebagai langkah-langkah sistematis untuk menyelesaikan suatu permasalahan secara logis dan terstruktur. Algoritma banyak digunakan dalam berbagai aplikasi seperti pencarian data, pengurutan, serta pengolahan informasi dalam skala kecil maupun besar.

Dalam penggunaannya, efisiensi algoritma menjadi hal yang sangat diperhatikan. Hal ini dikarenakan semakin besar data yang diproses, maka semakin dibutuhkan algoritma yang mampu bekerja dengan cepat dan menggunakan sumber daya secara optimal. Oleh sebab itu, diperlukan analisis kompleksitas algoritma untuk mengetahui tingkat efisiensi suatu algoritma, baik dari segi waktu proses maupun penggunaan memori.

Analisis kompleksitas algoritma mencakup pemahaman terhadap kompleksitas waktu (time complexity) dan kompleksitas ruang (space complexity). Selain itu, digunakan pula notasi matematika seperti Big O, Big Theta, dan Big Omega untuk menggambarkan performa algoritma dalam berbagai kondisi. Struktur dasar dalam algoritma seperti perulangan, percabangan, dan rekursi juga turut dianalisis untuk mengetahui pengaruhnya terhadap kinerja program secara keseluruhan.

Pemahaman mengenai analisis kompleksitas algoritma sangat penting bagi pengembang perangkat lunak agar dapat memilih dan merancang algoritma yang efisien. Dengan demikian, program yang dihasilkan tidak hanya berjalan dengan benar, tetapi juga memiliki performa yang baik dan mampu menangani data dalam jumlah besar secara efektif.

Melalui praktikum analisis kompleksitas algoritma, saya berharapkan dapat memahami konsep dasar analisis algoritma serta mampu mengaplikasikannya dalam berbagai kasus. Praktikum ini memberikan pengalaman dalam menganalisis, membandingkan, serta mengevaluasi efisiensi dari beberapa algoritma yang digunakan.Dengan mempelajari materi ini secara lebih mendalam, diharapkan pengguna memiliki kemampuan dalam menentukan algoritma yang tepat, serta mampu mengembangkan solusi yang efisien, optimal, dan sesuai dengan kebutuhan perkembangan teknologi saat ini

1.2 TujuanPraktikum

A. Memahami konsep dasar analisis kompleksitas algoritma.
B. Mengetahui perbedaan kompleksitas waktu (time complexity) dan kompleksitas ruang (space complexity).
C. Mempelajari penggunaan notasi Big O, Big Theta, dan Big Omega dalam analisis algoritma.
D. Mengembangkan kemampuan dalam merancang algoritma yang efisien dan terstruktur.

1.3 Manfaat Praktikum

A. Menambah pemahaman tentang efisiensi algoritma dalam pemrograman.
B. Membantu dalam memilih algoritma yang tepat sesuai dengan kebutuhan masalah.
C. Meningkatkan kemampuan berpikir logis dan analitis dalam menyelesaikan masalah.

BAB II
LANDASAN TEORI

2.1 Analisis Kompleksitas Algoritma

merupakan suatu proses yang digunakan untuk mengukur tingkat efisiensi sebuah algoritma dalam menyelesaikan suatu permasalahan. Dalam ilmu komputer, algoritma adalah langkah-langkah logis dan sistematis yang dirancang untuk menyelesaikan suatu tugas tertentu. Setiap algoritma memiliki tingkat performa yang berbeda-beda tergantung pada cara kerja dan jumlah data yang diproses. Oleh karena itu, analisis kompleksitas sangat penting untuk mengetahui seberapa baik suatu algoritma bekerja ketika digunakan pada berbagai kondisi input.

Analisis kompleksitas algoritma berfungsi untuk mengevaluasi kebutuhan sumber daya yang digunakan oleh sebuah program, terutama dari segi waktu eksekusi (time complexity) dan penggunaan memori (space complexity). Dengan melakukan analisis ini, programmer dapat menentukan algoritma mana yang lebih efisien dan lebih cocok digunakan untuk menyelesaikan suatu masalah, khususnya ketika berhadapan dengan data dalam jumlah besar. Algoritma yang baik bukan hanya menghasilkan output yang benar, tetapi juga mampu bekerja dengan cepat dan hemat memori.

Dalam analisis kompleksitas, biasanya digunakan notasi matematika seperti Big O Notation, Big Omega, dan Big Theta untuk menggambarkan pertumbuhan performa algoritma terhadap ukuran input. Big O digunakan untuk menunjukkan batas atas atau worst case, yaitu kondisi terburuk dari performa algoritma. Dengan notasi ini, programmer dapat memperkirakan seberapa besar peningkatan waktu proses ketika jumlah data bertambah.

Sebagai contoh, algoritma pencarian sederhana seperti linear search memiliki kompleksitas O(n), karena dalam kasus terburuk harus memeriksa seluruh elemen data satu per satu. Sedangkan binary search memiliki kompleksitas O(log n), yang jauh lebih efisien karena proses pencarian dilakukan dengan membagi data menjadi dua bagian secara terus-menerus. Perbedaan ini menunjukkan bahwa pemilihan algoritma yang tepat sangat berpengaruh terhadap performa program. Selain itu, analisis kompleksitas juga berkaitan erat dengan struktur data yang digunakan. Pemilihan struktur data seperti array, linked list, stack, queue, tree, dan graph dapat memengaruhi kecepatan akses data serta efisiensi algoritma secara keseluruhan. Struktur data yang tepat akan membantu algoritma bekerja lebih optimal dan menghasilkan solusi yang lebih efektif.

Dalam perkembangan teknologi saat ini, analisis kompleksitas algoritma menjadi salah satu bagian penting dalam pengembangan perangkat lunak modern. Sistem seperti mesin pencari, aplikasi mobile, database, kecerdasan buatan, dan

sistem jaringan membutuhkan algoritma yang cepat dan efisien agar dapat memberikan performa terbaik kepada pengguna. Oleh karena itu, pemahaman mengenai analisis kompleksitas algoritma sangat penting bagi mahasiswa maupun programmer yang ingin mendalami bidang pemrograman dan pengembangan sistem secara profesional.

2.2 Big O Notation

Big O Notation adalah notasi matematika yang digunakan untuk mengukur dan menggambarkan tingkat efisiensi suatu algoritma, terutama dalam kondisi terburuk (worst case). Big O menunjukkan bagaimana pertumbuhan waktu eksekusi atau penggunaan memori suatu algoritma ketika jumlah data atau input (n) semakin besar. Dengan menggunakan Big O, programmer dapat membandingkan performa beberapa algoritma tanpa harus menjalankannya secara langsung.

Sebagai contoh, algoritma dengan kompleksitas O(1) berarti waktu prosesnya tetap meskipun jumlah data bertambah. Algoritma dengan kompleksitas O(n) berarti waktu proses bertambah seiring bertambahnya jumlah input. Sedangkan $O(n^2)$ menunjukkan bahwa waktu proses meningkat jauh lebih cepat karena adanya perulangan bersarang (nested loop). Big O sangat penting dalam pemrograman karena membantu memilih algoritma yang lebih efisien dan optimal.

2.3 Time Complexity & Space Complexity

Time Complexity dan Space Complexity merupakan dua aspek utama dalam analisis kompleksitas algoritma yang digunakan untuk mengukur efisiensi sebuah program. Time Complexity digunakan untuk mengetahui seberapa lama waktu yang dibutuhkan sebuah algoritma untuk menyelesaikan suatu proses berdasarkan jumlah input yang diberikan. Time complexity tidak dihitung dalam satuan detik, melainkan berdasarkan jumlah langkah atau operasi yang dilakukan selama algoritma berjalan.

Sebagai contoh, jika sebuah algoritma harus memeriksa setiap elemen dalam array satu per satu, maka kompleksitas waktunya adalah O(n). Jika menggunakan dua perulangan bersarang, maka biasanya kompleksitas waktunya menjadi $O(n^2)$. Semakin kecil nilai time complexity, maka semakin cepat algoritma tersebut bekerja, terutama saat menangani data dalam jumlah besar.

Sedangkan Space Complexity digunakan untuk mengukur seberapa banyak memori yang dibutuhkan oleh algoritma selama proses eksekusi. Space complexity menghitung penggunaan variabel tambahan, array baru, struktur data sementara, maupun proses rekursi yang membutuhkan ruang penyimpanan tambahan.

Jika algoritma hanya menggunakan sedikit memori tambahan, maka kompleksitas ruangnya rendah. Dalam praktikum, programmer harus menyeimbangkan antara time complexity dan space complexity. Terkadang

algoritma yang lebih cepat membutuhkan memori yang lebih besar, sedangkan algoritma yang lebih hemat memori bisa membutuhkan waktu proses yang lebih lama.

2.4 Fungsi & Kegunaan

A. Mengukur efisiensi algoritma berdasarkan waktu proses dan penggunaan memori
B. Membantu memilih algoritma yang paling optimal untuk suatu permasalahan
C. Membandingkan performa beberapa algoritma yang memiliki tujuan sama
D. Menentukan apakah algoritma cocok digunakan pada data dalam jumlah besar
E. Mengurangi risiko program berjalan lambat atau tidak efisien
F. Menghindari pemborosan penggunaan memori komputer
G. Membantu perancangan sistem yang lebih cepat, stabil, dan efisien

BAB III
METODOLOGI PRAKTIKUM

3.1 Alat Dan Bahan

- PC/Laptop
- Koneksi internet (opsional)
- VScode (code editor)
- Git bash
- Github (untuk menyimpan project secara online)
- Javascript

3.2 Langkah – Langkah

1. Pastikan kamu berada di dalam folder praktikum-5. Buat file baru bernama **01-intro-kompleksitas.js.** lalu ketikkan code berikut

```javascript
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}
const N = 100_000; // ukuran data: 100 ribu

function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}

function jumlahkanRumus(n) {
  return (n * (n + 1)) / 2;
}

function cariPasangan(arr) {
  const pasangan = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
    }
  }
  return pasangan;
}
const data = Array.from({ length: 5000 }, (_, i) => i);
console.log("=== Perbandingan Waktu Eksekusi ===");
ukurWaktu("O(1) jumlahkanRumus ", () => jumlahkanRumus(N));
ukurWaktu("O(n) jumlahkanLinear ", () => jumlahkanLinear(N));
ukurWaktu("O(n2) cariPasangan ", () => cariPasangan(data));
console.log("\nHasil sama?", jumlahkanLinear(100) === jumlahkanRumus(100));
```

Simpan file dan Jalankan di git bash dan amati hasilnya

```
PS C:\2025573010141_algoritma_struktur_data\praktikum-5> node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 3 ms
O(n2) cariPasangan : 51 ms

Hasil sama? true
```

Penjelasan dari code tersebut:

analisis kompleksitas algoritma sangat penting untuk mengetahui tingkat efisiensi suatu program. Pada kode tersebut dilakukan perbandingan waktu eksekusi antara algoritma dengan kompleksitas O(1), O(n), dan O(n^2), di mana hasilnya menunjukkan bahwa algoritma dengan kompleksitas O(1), yaitu jumlahkanRumus(), memiliki waktu eksekusi paling cepat karena hanya menggunakan satu proses perhitungan langsung tanpa perulangan, sedangkan algoritma dengan kompleksitas O(n), yaitu jumlahkanLinear(), membutuhkan waktu lebih lama karena melakukan perulangan sebanyak jumlah input yang diberikan, dan algoritma dengan kompleksitas O(n^2), yaitu cariPasangan(), memiliki waktu eksekusi paling lama karena menggunakan perulangan bersarang (nested loop) yang membuat jumlah proses meningkat sangat besar seiring bertambahnya data.

2. Buat file **02- kelas-big-o.js** di dalam folder praktikum-5.lalu ketikkan code berikut,simpan file dan jalankan di terminal.

```javascript
function ambilPertama(arr) {
    return arr[0];
}
function tambahkanItem(arr, item) {
    arr.push(item);
}
function isGenap(n) {
    return n % 2 === 0;
}

function binarySearch(arr, target) {
    let kiri = 0,
        kanan = arr.length - 1;
    let langkah = 0;
    while (kiri <= kanan) {
        langkah++;
        const tengah = Math.floor((kiri + kanan) / 2);
        if (arr[tengah] === target) {
            console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
            return tengah;
        }
        if (arr[tengah] < target) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    return -1;
}
function cariMax(arr) {
    let maks = arr[0];
    for (const x of arr) if (x > maks) maks = x;
    return maks;
}
// ─── O(n2) — Kuadratik ──────────────────────────────
function bubbleSortNaif(arr) {
    const a = [...arr];
    for (let i = 0; i < a.length; i++)
        for (let j = 0; j < a.length - i - 1; j++)
            if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
    return a;
}
// ─── O(2n) — Eksponensial ───────────────────────────
function fibRekursif(n) {
    if (n <= 1) return n;
    return fibRekursif(n - 1) + fibRekursif(n - 2); // dua panggilan rekursif!
}
// ── Demonstrasi ──────────────────────────────────────
console.log("=== O(1) — selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42)); // true
console.log("\n=== O(log n) — Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452); // cari di 1 juta data
console.log("\n=== O(n) — Linear Search ===");
console.log(
    "Max dari 1000 elemen:",
    cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
);
console.log("\n=== O(n2) — Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
console.log("\n=== O(2n) — Fibonacci Rekursif (n kecil!) ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
console.log("");
// Demonstrasi betapa lambatnya O(2n)
console.log("\nWaktu fib(35) O(2n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");
console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
t = Date.now();
fibMemo(35);
console.log(Date.now() - t, "ms");
```

Ouputnya:

```
PS C:\2025573010141_algoritma_struktur_data\praktikum-5> node 02-kelas-big-0.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 999

=== O(n2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
190 ms
Waktu fib(35) O(n) memoization:
0 ms
PS C:\2025573010141_algoritma_struktur_data\praktikum-5>
```

Berikut kelas-kelas Big O dari yang paling cepat ke paling lambat:

- O(1) Konstan. Waktu eksekusi tidak bergantung pada ukuran input sama sekali.
- Contoh: akses elemen array dengan indeks, operasi aritmatika, akses property object.
- O(log n) — Logaritmik. Setiap langkah memotong masalah menjadi separuh.
- Contoh: Binary Search — mencari di 1 juta data hanya butuh ~20 langkah.
- O(n) — Linear. Waktu tumbuh sebanding lurus dengan ukuran input.
- Contoh: loop sekali melewati array, Linear Search.
- O(n log n) — Linearitmik. Sedikit lebih buruk dari linear.
- Contoh: Merge Sort, Quick Sort — algoritma sorting terbaik yang bisa dicapai.
- $O(n^2)$ — Kuadratik. Nested loop — sangat lambat untuk data besar.
- Contoh: Bubble Sort, Selection Sort, nested loop dua tingkat.
- $O(2^n)$ — Eksponensial. Sangat buruk —hanya bisa digunakan untuk n sangat kecil.
- Contoh: Fibonacci rekursif naif, subset generation.

3. **03-space-complexity.js/Latihan Class Bank Account**
   ketikkan code berikut dan latihan sesuai instruksi di modul praktikum,lalu jalankan di terminal dengan prompt”node 03-space.complexity.js

```javascript
// --- O(1) Space: hanya pakai variabel tambahan konstan ---
function jumlahArray(arr) {
    let total = 0; // hanya 1 variabel tambahan
    for (const x of arr) total += x;
    return total;
}
// --- O(n) Space: membuat struktur baru sebesar input ---
function duplikasiArray(arr) {
    const baru = []; // array baru tumbuh seiring arr
    for (const x of arr) baru.push(x * 2);
    return baru;
}
// --- O(n) Space: rekursi (call stack) ---
function faktorialRekursif(n) {
    if (n <= 1) return 1;
    return n * faktorialRekursif(n - 1); // n frame di call stack
}
// --- O(1) Space: faktorial iteratif ---
function faktorialIteratif(n) {
    let hasil = 1;
    for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
    return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log('Jumlah array :', jumlahArray(arr)); // O(1) space
console.log('Duplikasi array:', duplikasiArray(arr)); // O(n) space
console.log('Faktorial 10 rekursif :', faktorialRekursif(10));
console.log('Faktorial 10 iteratif :', faktorialIteratif(10));
// Visualisasi: hitung elemen unik (O(n) space)
function hitungUnik(arr) {
    const seen = new Set(); // Set tumbuh hingga n elemenfor (const x of arr) seen.add(x);
    return seen.size;
}
const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log('Elemen unik:', hitungUnik(dataAcak)); // 7


// 1. METODE LAMBAT (NESTED LOOP)
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}

// Big O Time: O(n²)
// Alasan: Nested loop → bandingkan semua pasangan
// Big O Space: O(1)
// Alasan: Tidak pakai struktur data tambahan

// 2. METODE CEPAT (SET)

function cariPasanganCepat(arr, target) {
  const seen = new Set();

  for (let num of arr) {
    const pasangan = target - num;

    if (seen.has(pasangan)) {
      return [pasangan, num];
    }

    seen.add(num);
  }

  return null;
}
```

```javascript
// latihan Class bank//
// Big O Time: O(n)
// Alasan: Hanya 1 loop
// Big O Space: O(n)
// Alasan: Menyimpan data di Set

// 3. TEST CASE KECIL

const arrTest = [2, 7, 11, 15];
const targetTest = 9;

console.log("=== TEST KECIL ===");
console.log("Lambat:", cariPasanganLambat(arrTest, targetTest));
console.log("Cepat:", cariPasanganCepat(arrTest, targetTest));

// 4. FUNGSI UKUR WAKTU

function ukurWaktu(fn, arr, target) {
  const start = Date.now();
  const result = fn(arr, target);
  const end = Date.now();

  console.log(`${fn.name} → waktu: ${end - start} ms, hasil:`, result);
}

// 5. TEST BESAR (50.000 DATA)

const ukuran = 50000;

// buat array random
const arrBesar = Array.from({ length: ukuran }, () =>
  Math.floor(Math.random() * 100000),
);

// target yang hampir pasti tidak ada
const targetBesar = -1;

console.log("\n=== TEST BESAR ===");

// WARNING
ukurWaktu(cariPasanganCepat, arrBesar, targetBesar);

```

Output:

```
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 0
=== TEST KECIL ===
Lambat: [ 2, 7 ]
Cepat: [ 2, 7 ]

=== TEST BESAR ===
cariPasanganCepat → waktu: 14 ms, hasil: null
```

Berdasarkan program tersebut fungsi yang lebih baik tergantung pada kebutuhan dan kondisi penggunaan. Pada pencarian pasangan angka, fungsi cariPasanganCepat() lebih baik dari segi waktu karena memiliki kompleksitas **O(n)**, sedangkan cariPasanganLambat() memiliki kompleksitas **O(n^2)**. Hal ini terjadi karena cariPasanganCepat() hanya menggunakan satu perulangan dan bantuan struktur data Set, sehingga proses pencarian menjadi jauh lebih cepat terutama ketika jumlah data sangat besar. Sebaliknya, cariPasanganLambat() menggunakan nested loop yang membandingkan semua kemungkinan pasangan satu per satu, sehingga waktu eksekusinya jauh lebih lama.

Selain itu kita harus mempertimbangkan memori yang digunakan algoritma. Inilah yang disebut Space Complexity.

Space complexity mengukur memori tambahan (auxiliary space) yang dibutuhkan algoritma
seiring bertambahnya input — tidak termasuk input itu sendiri.

Contoh perbandingan:

▪Algoritma yang hanya menggunakan beberapa variabel → O(1) space

▪Algoritma yang membuat array baru sebesar n → O(n) space

▪Rekursi sedalam n tingkat → O(n) space (call stack)

Trade-off yang umum: Banyak algoritma bisa dibuat lebih cepat dengan mengorbankan lebih
banyak memori, atau sebaliknya. Ini disebut time-space trade-off.

4. **04-Refactor.js** ketikkan code berikut dan jalankan di terminal.dengan prompt" node 04-refactor.js"

```javascript
function adaduplikatlambat(arr) {
    const seen = new Set();
    for (const x of arr) {
        if (seen.has(x)) return true;
    }
    return false;
}
function frekuensiLambat(arr) {
    const unik = [];
    const hitung = [];
    for (const x of arr) {
        const idx = unik.indexOf(x);
        if (idx === -1) {
            unik.push(x);
            hitung.push(1);
        } else hitung[idx]++;
    }
    return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}
function frekuensiCepat(arr) {
    const counter = {};
    for (const x of arr) counter[x] = (counter[x] || 0) + 1;
    return counter;
}

const besar = Array.from({ length: 20000 }, () =>
    Math.floor(Math.random() * 1000),
);

let t = Date.now();
adaduplikatlambat(besar);
console.log('duplikat CEPAT 0(n):', Date.now() - t, 'ms');

const kata = ['a', 'b', 'a', 'c', 'b', 'a', 'd'];
console.log('\nfrekuensi :', frekuensiCepat(kata));
```

Output :

```
PS C:\2025573010141_algoritma_struktur_data\praktikum-5> node 04-refactor.js
duplikat CEPAT 0(n): 3 ms

frekuensi : { a: 3, b: 2, c: 1, d: 1 }
PS C:\2025573010141_algoritma_struktur_data\praktikum-5>
```

fungsi frekuensiLambat memiliki kompleksitas $O(n^2)$ sehingga kurang efisien, sedangkan frekuensiCepat lebih optimal dengan kompleksitas $O(n)$. Penggunaan struktur data seperti hash map atau object terbukti dapat meningkatkan performa algoritma. Oleh karena itu, analisis kompleksitas sangat penting dalam memilih algoritma yang paling efektif dan efisien.

Tabel Referensi

Pola Kode	Kompleksitas	Contoh Nyata
Akses array/object langsung	$O(1)$	arr[0], obj.key, Map.get()
Satu loop melewati n elemen	$O(n)$	forEach, for loop linear
Loop i*=2 atau i/=2	$O(\log n)$	Binary search, halving loop
Dua loop berurutan (bukan nested)	$O(n)$	$2n \rightarrow O(n)$ setelah buang konstanta
Dua loop bersarang (nested)	$O(n^2)$	Bubble sort, cari pasangan naif
Tiga loop bersarang	$O(n^3)$	Jarang tapi ada di algoritma matrix
Rekursi bercabang dua	$O(2^n)$	Fibonacci naif, power set
Rekursi + sorting	$O(n \log n)$	Merge sort, quick sort rata-rata

5. **Latihan-1.js identifikasi Kompleksitas**

```javascript
function hitungKompleksitas(n, fn) {
  const start = Date.now();
  fn(n);
  const end = Date.now();
  console.log(`Waktu eksekusi untuk n=${n}: ${end - start} ms`);
}

// Fungsi A
function fnA(n) {
  return n * 2;
}
// Big O: O(1)
// Alasan: Operasi tetap, tidak bertambah walaupun n besar

// Fungsi B

// Nested loop (2 lapis)
function fnB(n) {
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      console.log(i, j);
    }
  }
}
// Big O: O(n^2)
// Alasan: Loop dalam loop → n * n = n²

// Fungsi C
function fnC(n) {
  for (let i = 1; i < n; i *= 2) {
    console.log(i);
  }
}
// Big O: O(log n)
// Alasan: Pertumbuhan eksponensial (1 → 2 → 4 → 8 ...)

// Fungsi D
function fnD(n) {
  const arr = Array.from({ length: n }, (_, i) => i);

  arr.forEach((x) => {
    arr.forEach((y) => {
      arr.forEach((z) => {
        console.log(x, y, z);
      });
    });
  });
}
// Big O: O(n^3)
// Alasan: 3 loop bersarang → n * n * n = n³

const n = 1000;

console.log("=== Fungsi A ===");
hitungKompleksitas(n, fnA);

console.log("=== Fungsi B ===");

hitungKompleksitas(n, fnB);

console.log("=== Fungsi C ===");
hitungKompleksitas(n, fnC);

console.log("=== Fungsi D ===");
```

Output:

```
999 999
Waktu eksekusi untuk n=1000: 115721 ms
=== Fungsi C ===
1
2
4
8
16
32
64
128
256
512
Waktu eksekusi untuk n=1000: 0 ms
=== Fungsi D ===
```

6. **Tugas-1.js**

```javascript
function intersectionN2(arr1, arr2) {
  // Time: O(n^2), Space: O(1)
  const result = [];
  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j] && !result.includes(arr1[i])) {
        result.push(arr1[i]);
      }
    }
  }
  return result;
}

// Versi O(n) menggunakan Set
function intersectionN(arr1, arr2) {
  // Time: O(n), Space: O(n)
  const set2 = new Set(arr2);
  const result = [];

  for (let item of arr1) {
    if (set2.has(item)) {
      result.push(item);
    }
  }

  return [...new Set(result)]; // hapus duplikat
}

// FUNGSI B - KELOMPOK ANAGRAM

function groupAnagrams(words) {
  // Time: O(n log k), Space: O(n)
  // n = jumlah kata, k = panjang kata

  const map = new Map();

  for (let word of words) {
    const key = word.split("").sort().join("");

    if (!map.has(key)) {
      map.set(key, []);
    }

    map.get(key).push(word);
  }

  return Array.from(map.values());
}

// FUNGSI C - SUM = KUADRAT

// Versi O(n^3)
function sumSquareN3(arr) {
  // Time: O(n^3), Space: O(1)
  const n = arr.length;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      for (let k = 0; k < n; k++) {
        if (arr[i] + arr[j] === arr[k] * arr[k]) {
          return true;
        }
      }
    }
  }
  return false;
}

// Versi O(n log n)
function sumSquareOptimized(arr) {
  // time: O(n log n), Space: O(1)
  arr.sort((a, b) => a - b);

  const n = arr.length;

  for (let k = n - 1; k >= 0; k--) {
    let left = 0;
    let right = k - 1;

    const target = arr[k] * arr[k];

    while (left <= right) {
      const sum = arr[left] + arr[right];

      if (sum === target) {
        return true;
      } else if (sum < target) {
        left++;
      } else {
        right--;
      }
    }
  }

  return false;
}

// TEST & PENGUKURAN WAKTU

// Data besar
const size = 5000;
const arr1 = Array.from({ length: size }, () =>
  Math.floor(Math.random() * 10000),
);
const arr2 = Array.from({ length: size }, () =>
  Math.floor(Math.random() * 10000),
);

const words = ["eat", "tea", "tan", "ate", "nat", "bat"];

// Fungsi A test
console.time("Intersection O(n^2)");
intersectionN2(arr1, arr2);
console.timeEnd("Intersection O(n^2)");

console.time("intersection O(n)");
intersectionN(arr1, arr2);
console.timeEnd("Intersection O(n)");

// Fungsi B test
console.log("Anagram:", groupAnagrams(words));

// Fungsi C test
console.time("SumSquare O(n^3)");
sumSquareN3([1, 2, 3, 4, 5, 6, 7, 8, 9]);
console.timeEnd("SumSquare O(n^3)");

console.time("SumSquare O(n log n)");
sumSquareOptimized([1, 2, 3, 4, 5, 6, 7, 8, 9]);
console.timeEnd("SumSquare O(n log n)");
```

Output :

```
PS C:\2025573010141_algoritma_struktur_data\praktikum-5> cd tugas
PS C:\2025573010141_algoritma_struktur_data\praktikum-5\tugas> node tugas-1.js
Intersection O(n^2): 87.671ms
Intersection O(n): 2.799ms
Anagram: [ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
SumSquare O(n^3): 0.161ms
SumSquare O(n log n): 0.251ms
PS C:\2025573010141_algoritma_struktur_data\praktikum-5\tugas>
```

7. **Tugas-2.js**

```javascript
// O(1)
function fn_O1(n) {
  // Time: O(1), Space: O(1)
  return n + 1;
}

// O(n)
function fn_On(n) {
  // Time: O(n), Space: O(1)
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += i;
  }
  return sum;
}

// O(n log n)
function fn_OnLogn(n) {
  // Time: O(n log n), Space: O(1)
  let count = 0;
  const logN = Math.floor(Math.log2(n));

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < logN; j++) {
      count++;
    }
  }
  return count;
}

// O(n^2)
function fn_On2(n) {
  // Time: O(n^2), Space: O(1)
  let count = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      count++;
    }
  }
  return count;
}

// BENCHMARK FUNCTION
function benchmarkSemua(ukuranData) {
  console.log("===== HASIL BENCHMARK =====");

  for (let n of ukuranData) {
    console.log(`\nUkuran data: ${n}`);

    console.time("O(1)");
    fn_O1(n);
    console.timeEnd("O(1)");

    console.time("O(n)");
    fn_On(n);
    console.timeEnd("O(n)");

    console.time("O(n log n)");
    fn_OnLogn(n);
    console.timeEnd("O(n log n)");

    console.time("O(n^2)");
    fn_On2(n);
    console.timeEnd("O(n^2)");
  }
}

benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

Output :

```
PS C:\2025573010141_algoritma_struktur_data\praktikum-5\tugas> node tugas-2.js
===== HASIL BENCHMARK =====

Ukuran data: 100
O(1): 0.061ms
O(n): 0.092ms
O(n log n): 0.167ms
O(n^2): 0.527ms

Ukuran data: 500
O(1): 0.038ms
O(n): 0.036ms
O(n log n): 0.295ms
O(n^2): 2.065ms

Ukuran data: 1000
O(1): 0.021ms
O(n): 0.192ms
O(n log n): 0.387ms
O(n^2): 7.031ms

Ukuran data: 5000
O(1): 0.015ms
O(n): 0.045ms
O(n log n): 0.378ms
O(n^2): 96.606ms

Ukuran data: 10000
O(1): 0.008ms
O(n): 0.049ms
O(n log n): 0.456ms
O(n^2): 254.701ms
PS C:\2025573010141_algoritma_struktur_data\praktikum-5\tugas>
```

BAB IV
PENUTUP

4.1 KESIMPULAN

Berdasarkan praktikum yang telah saya lakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma merupakan bagian penting dalam mempelajari algoritma dan struktur data karena membantu dalam menentukan tingkat efisiensi suatu program. Pada praktikum ini, saya mempelajari bagaimana cara menganalisis performa algoritma berdasarkan waktu eksekusi (time complexity) dan penggunaan memori (space complexity). Saya juga memahami penggunaan Big O Notation sebagai alat untuk menggambarkan tingkat pertumbuhan suatu algoritma ketika jumlah data semakin besar. Selain itu, saya mempelajari perbedaan antara algoritma dengan kompleksitas O(1), O(n), dan $O(n^2)$ melalui beberapa contoh program seperti penjumlahan dengan rumus, penjumlahan menggunakan perulangan, serta pencarian pasangan angka menggunakan nested loop dan Set. Dari hasil tersebut, saya dapat melihat bahwa algoritma dengan kompleksitas yang lebih kecil memiliki performa yang lebih baik dan lebih efisien dalam menangani data besar.

4.2 SARAN

terdapat beberapa saran yang diharapkan dapat meningkatkan pemahaman dan kualitas pembelajaran di masa mendatang:

- Perbanyak latihan dalam membuat dan menganalisis berbagai jenis algoritma agar lebih memahami konsep time complexity dan space complexity secara nyata.
- Pahami terlebih dahulu dasar-dasar algoritma dan struktur data seperti array, linked list, stack, queue, serta perulangan sebelum mempelajari analisis kompleksitas agar tidak mengalami kesulitan saat praktik.
- Biasakan menggunakan Big O Notation dalam setiap analisis program sehingga dapat lebih mudah menentukan tingkat efisiensi suatu algoritma.
- Gunakan berbagai contoh kasus seperti searching, sorting, rekursi, dan penggunaan struktur data tambahan agar pemahaman terhadap kompleksitas algoritma menjadi lebih mendalam.